UNIVERSIDADE DE RIO VERDE
FACULDADE DE ENGENHARIA DE SOFTWARE

# ATIVIDADE DOMICILIAR

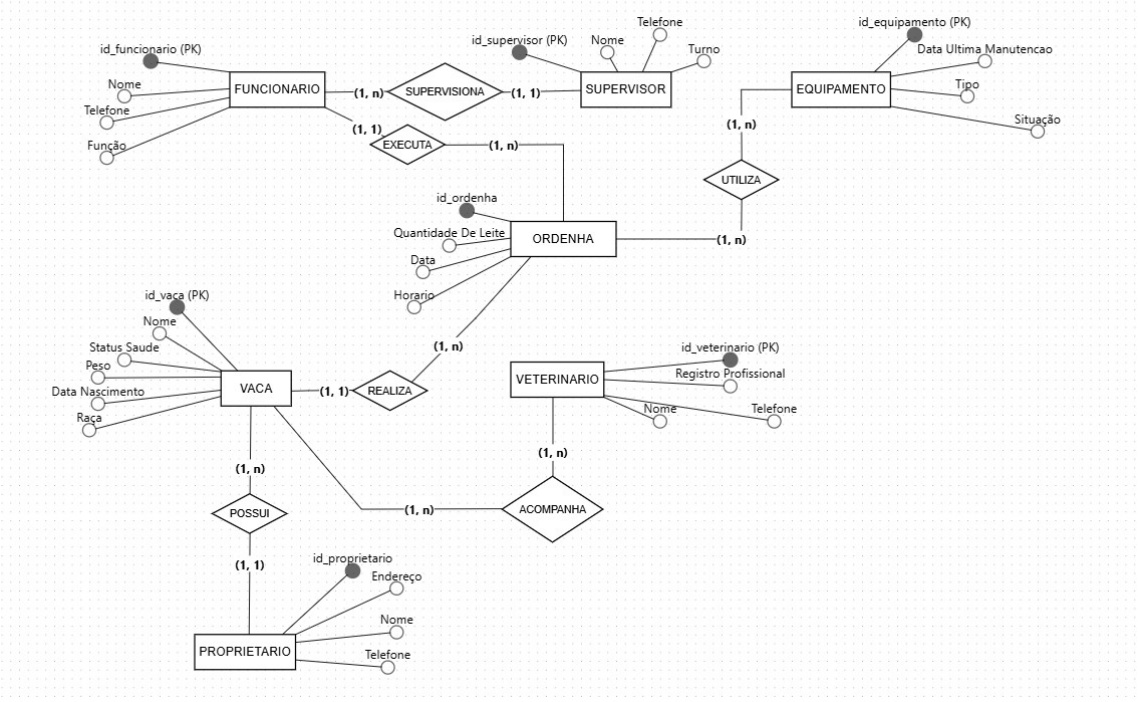
WAYSER HENRIQUE DE FARIA LOPES
Prof. SÉRGIO SOUZA NOVAK

Tarefas domiciliares aceitas para abono de faltas na disciplina ESW417 BANCO DE DADOS, durante o período de 09/03/2026 a 13/03/2026

**RIO VERDE – GO**
**2026**

**ACDEMICO:** *Wayser Henrique de Faria Lopes*

***DER DE SISTEMA DE ORDENHA***



# Lista de Exercícios – Banco de Dados

**Acedmico:** *Wayser Henrique de Faria Lopes*

## Exercício 01 – PetShop 2.0

a) Não seria possível inserir o id_animal = 30, porque esse animal não existe na tabela ANIMAL. Isso quebraria a integridade referencial, já que a chave estrangeira precisa apontar para um registro válido.

b) O cpf_sup do Ricardo aponta para outro gerente da mesma tabela, que é a Ana Costa. Isso mostra uma hierarquia, onde um gerente pode supervisionar outro gerente.

c) O CPF do João aparece nas duas tabelas porque Veterinário é uma especialização de Pessoa. Isso mostra uma estratégia de herança, onde a pessoa já existe e só ganha novos dados.

d) Um exemplo de relacionamento 1:N é entre PESSOA e ANIMAL, pois uma pessoa pode ter vários animais. A chave estrangeira cpf_cliente mantém essa ligação.

## Exercício 02 – Autopeças

a) A chave primária composta da tabela ESTOQUE_ITEM é formada por part_num e id_prat. Não dá para usar só part_num porque a mesma peça pode estar em várias prateleiras.

b) Não é erro a peça P-001 aparecer duas vezes. Isso é normal em um relacionamento N:N, pois a peça pode estar em mais de uma prateleira.

c) Se a peça P-002 fosse importada, ela deveria entrar na tabela PECA_IMPORTADA. A integridade referencial garante que ela já exista na tabela PECA antes disso.

d) A quantidade é um atributo do relacionamento entre peça e prateleira, por isso fica na tabela ESTOQUE_ITEM e não na tabela PECA.

Desafio: Se a prateleira B5 fosse apagada com ON DELETE RESTRICT, a exclusão não seria permitida enquanto existirem peças vinculadas a ela.

**Exercício 03 – Hospital**

a) O erro é ter um paciente com chave primária NULL. Isso não é permitido, pois toda tabela precisa ter uma PK válida.

b) A especialidade do médico não deveria aceitar NULL. No projeto lógico, esse campo deveria ser definido como NOT NULL.

c) O id_paciente = 500 não existe na tabela PACIENTE e o crm_medico = CRM-888 não existe na tabela MEDICO. Isso gera chaves estrangeiras órfãs.

d) Não é permitido repetir o mesmo valor em uma chave primária. Isso quebra a unicidade da tabela.

Correção: A tabela INTERNACAO deveria ter FKs com constraints para garantir que só pacientes e médicos existentes possam ser usados.

**Exercício 04 – Localização da FK**

1) Biblioteca: A FK fica na tabela LIVRO apontando para EDITORA, pois é um relacionamento 1:N.

2) Gestão Pública: A FK deve ficar em MUNICÍPIO apontando para PREFEITO, pois todo município tem prefeito e evita valores NULL.

3) E-commerce: A FK fica na tabela PEDIDO apontando para CLIENTE.

4) Escolar: Esse relacionamento N:N é resolvido com uma tabela associativa de matrícula.

5) RH: A FK deve ficar na tabela DEPARTAMENTO apontando para EMPREGADO, pois todo departamento precisa ter um gerente.

UNIRV - Banco de Dados

Lista de Exercícios – Respostas

EXERCÍCIO 02

a) A tabela ESTOQUE_ITEM possui uma chave primária composta formada pelos campos:

part_num + id_prat.

Isso acontece porque uma peça pode estar em várias prateleiras e uma prateleira pode armazenar várias peças. Portanto, é necessário usar os dois campos juntos para identificar unicamente cada registro.

b) A peça P-001 aparecer duas vezes não é erro. Isso ocorre porque o relacionamento entre PECA e PRATELEIRA é do tipo N:N (muitos para muitos). Assim, uma mesma peça pode estar em mais de uma prateleira.

c) Se a peça P-002 fosse importada, os dados adicionais deveriam ser inseridos na tabela PECA_IMPORTADA. Primeiro a peça precisa existir na tabela PECA e depois pode ser classificada como importada através da chave estrangeira part_num. Isso garante a integridade referencial.

d) O atributo "quantidade" representa um atributo do relacionamento entre PECA

e PRATELEIRA. Por isso ele está armazenado na tabela associativa ESTOQUE_ITEM.

Ele não poderia estar na tabela PECA porque a quantidade depende da prateleira onde a peça está armazenada.

Desafio Prático:

Se a prateleira B5 fosse apagada com a restrição ON DELETE RESTRICT,

o banco de dados não permitiria a exclusão porque existem registros

na tabela ESTOQUE_ITEM que dependem dela. Primeiro seria necessário

remover esses registros.

EXERCÍCIO 03

a) Violação de Integridade de Entidade:

Na tabela PACIENTE aparece um registro com id_paciente = NULL.
Isso é um erro porque uma chave primária nunca pode ser nula,
pois ela identifica unicamente cada registro da tabela.
b) Violação de Integridade de Domínio:
Na tabela MEDICO, a Dra. Beatriz possui especialidade = NULL.
Se todo médico deve possuir especialidade, o campo deveria
ter sido definido com a restrição NOT NULL.
c) Violação de Integridade Referencial:
Na tabela INTERNACAO:
Linha 2: id_paciente = 500 ® não existe na tabela PACIENTE.
Linha 3: crm_medico = CRM-888 ® não existe na tabela MEDICO.
Esses casos representam chaves estrangeiras órfãs, ou seja,
valores que referenciam registros que não existem nas tabelas
de origem.
d) Redundância e Chaves:
Na tabela MEDICO o CRM-123 aparece duas vezes. Isso não é permitido
quando o campo é definido como chave primária, pois a chave primária
deve ser única para cada registro.

EXERCÍCIO 04

1) Cenário Biblioteca:
Uma editora pode publicar vários livros, mas cada livro pertence a
apenas uma editora. Portanto, a chave estrangeira fica na tabela LIVRO.
LIVRO ® id_editora (FK)
2) Cenário Gestão Pública:
Cada município possui um prefeito em exercício.
MUNICIPIO ® id_prefeito (FK)
3) Cenário E-commerce:

Um cliente pode fazer vários pedidos, mas cada pedido pertence a um único cliente.

PEDIDO ® id_cliente (FK)

4) Cenário Escolar:

Um aluno pode se matricular em várias disciplinas e uma disciplina

pode ter vários alunos.

Relacionamento N:N resolvido com uma tabela associativa:

MATRICULA

- id_aluno

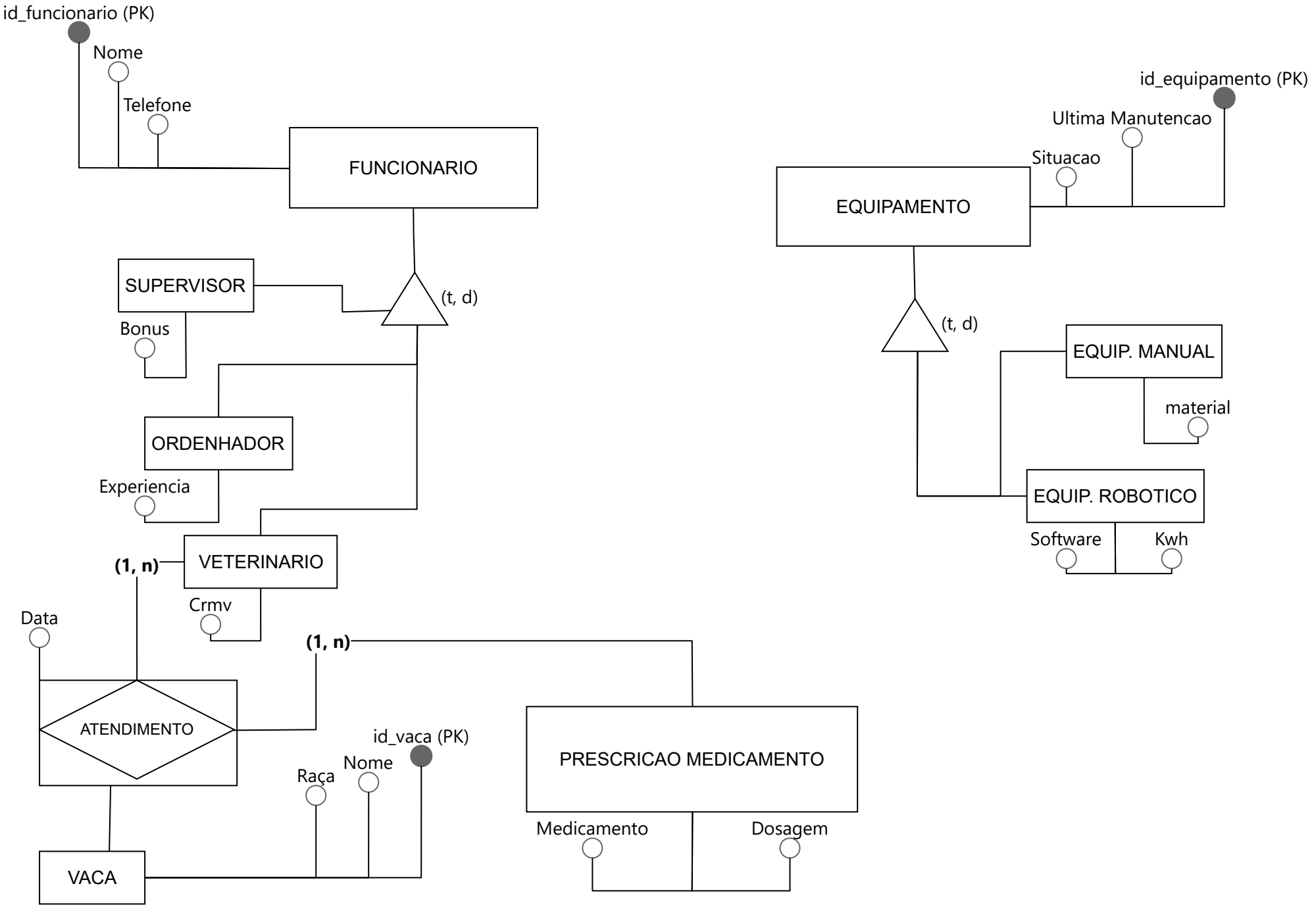
- id_disciplina

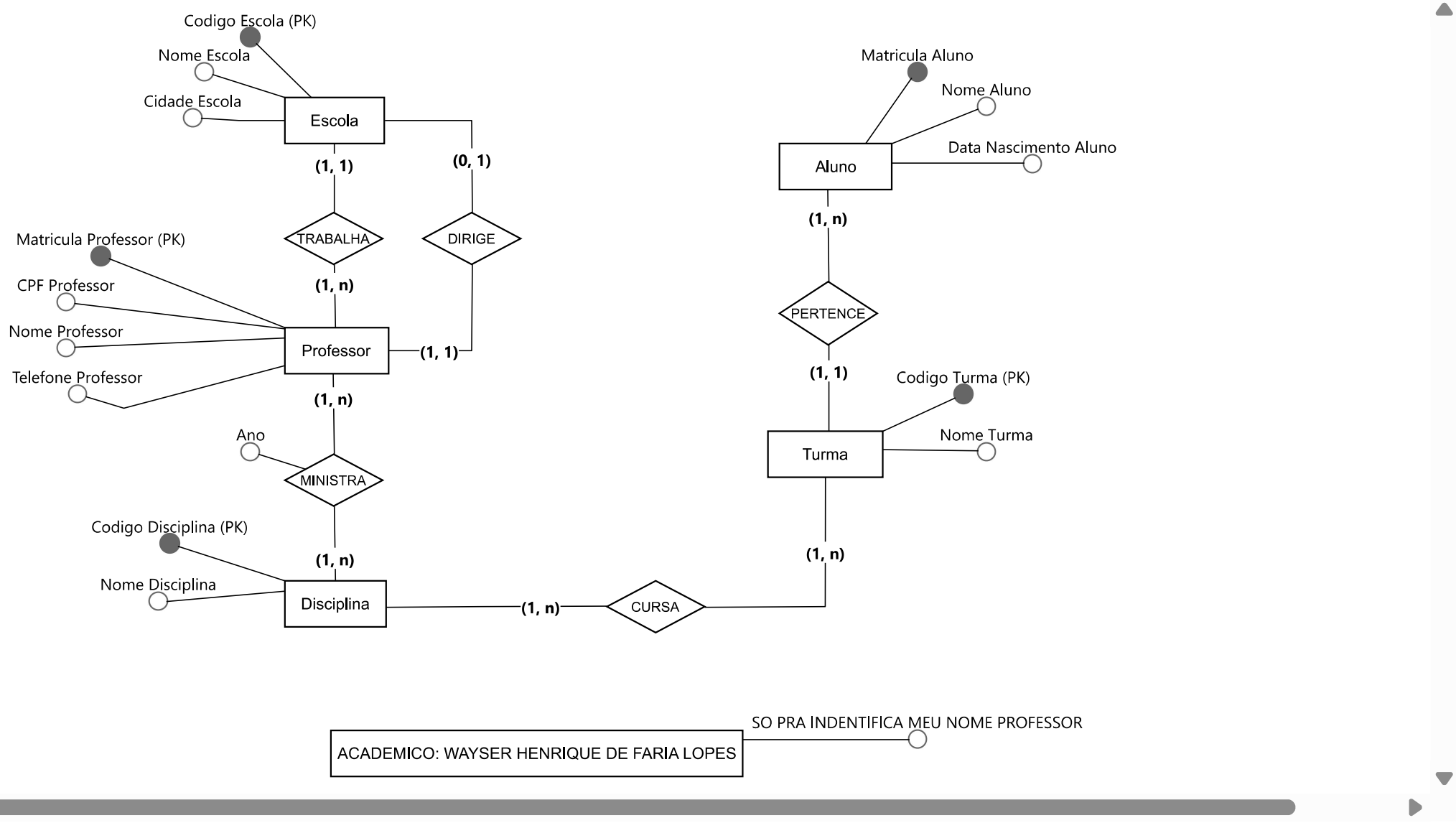
5) Cenário Recursos Humanos:

Cada departamento possui um gerente e cada gerente gerencia apenas

um departamento.

DEPARTAMENTO ® id_gerente (FK)


id_funcionario (PK)
Nome
Telefone
FUNCIONARIO
SUPERVISOR
(t, d)
Bonus
ORDENHADOR
Experiencia
VETERINARIO
(1, n)
Crmv
Data
(1, n)
ATENDIMENTO
id_vaca (PK)
Nome
Raça
VACA
PRESCRICAO MEDICAMENTO
Medicamento
Dosagem
id_equipamento (PK)
Ultima Manutencao
Situacao
EQUIPAMENTO
(t, d)
EQUIP. MANUAL
material
EQUIP. ROBOTICO
Software
Kwh


Codigo Escola (PK)
Nome Escola
Cidade Escola
Escola
(1, 1)
(0, 1)
TRABALHA
DIRIGE
Matricula Professor (PK)
CPF Professor
Nome Professor
Telefone Professor
Professor
(1, n)
(1, 1)
(1, n)
Ano
MINISTRA
(1, n)
Codigo Disciplina (PK)
Nome Disciplina
Disciplina
(1, n)
CURSA
Matricula Aluno
Nome Aluno
Data Nascimento Aluno
Aluno
(1, n)
PERTENCE
(1, 1)
Codigo Turma (PK)
Nome Turma
Turma
(1, n)
ACADEMICO: WAYSER HENRIQUE DE FARIA LOPES
SO PRA INDENTIFICA MEU NOME PROFESSOR

# Lista de Exercícios: Modelagem e SQL (DDL + DML)

*Instruções: Desenvolva os scripts completos utilizando PostgreSQL. Todas as tabelas devem conter chave primária, tipos de dados adequados e pelo menos uma restrição de integridade.*

## Objetivo da Atividade

Nesta lista você deverá:

- Criar tabelas utilizando o comando `CREATE TABLE`;
- Definir tipos de dados coerentes;
- Aplicar `PRIMARY KEY`, `NOT NULL`, `UNIQUE` e `DEFAULT`;
- Inserir dados utilizando `INSERT`;
- Realizar consultas com `SELECT`, `WHERE` e `ORDER BY`.

**1.** Crie a tabela **Professor** contendo:

- id (chave primária)
- nome (obrigatório)
- cpf (único e obrigatório)
- titulacao
- salario (decimal com 2 casas)
- ativo (boolean com valor padrão TRUE)

```
CREATE TABLE Professor (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) NOT NULL UNIQUE,
    titulacao VARCHAR(50),
    salario DECIMAL(10,2) CHECK (salario > 0),
    ativo BOOLEAN DEFAULT TRUE
);
```

**2.** Crie a tabela **Veterinario** contendo:

- id (chave primária)
- nome (obrigatório)
- crmv (registro profissional único)
- especialidade
- salario
- data_contratacao

```
CREATE TABLE Veterinario (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    crmv VARCHAR(20) NOT NULL UNIQUE,
    especialidade VARCHAR(50),
    salario DECIMAL(10,2),
    data_contratacao DATE
);
```

**3.** Crie a tabela **FornoIndustrial** contendo:

- id (chave primária)
- modelo
- temperatura_maxima
- capacidade_litros
- data_instalacao
- ativo (boolean)

```
CREATE TABLE FornoIndustrial (
    id SERIAL PRIMARY KEY,
    modelo VARCHAR(50),
    temperatura_maxima INTEGER CHECK
(temperatura_maxima > 0),
    capacidade_litros INTEGER,
    data_instalacao DATE,
    ativo BOOLEAN
);
```

**4.** Crie a tabela **Caldeira** contendo:

- id (chave primária)
- fabricante
- pressao_maxima
- volume
- data_ultima_inspecao

```
CREATE TABLE Caldeira (
    id SERIAL PRIMARY KEY,
    fabricante VARCHAR(50),
    pressao_maxima INTEGER CHECK
(pressao_maxima > 0),
    volume INTEGER,
    data_ultima_inspecao DATE
);
```

**5.** Crie a tabela **Drone** contendo:

- id (chave primária)
- modelo
- autonomia_minutos
- alcance_km
- peso
- ativo (boolean com default TRUE)

```
CREATE TABLE Drone (
    id SERIAL PRIMARY KEY,
    modelo VARCHAR(50),
    autonomia_minutos INTEGER CHECK
(autonomia_minutos > 0),
    alcance_km INTEGER,
    peso DECIMAL(6,2),
    ativo BOOLEAN DEFAULT TRUE
);
```

## Parte II – Inserção de Dados

Insira pelo menos **3 registros em cada tabela**.

**6.** Escreva os comandos `INSERT` da tabela Professor.

```
INSERT INTO Professor (nome, cpf, titulacao, salario)
VALUES
('Carlos Silva', '111.111.111-11', 'Doutorado', 7500.00),
('Mariana Souza', '222.222.222-22', 'Mestrado', 4800.00),
('João Pereira', '333.333.333-33', 'Especialista', 5200.00);
```

**7.** Escreva os comandos `INSERT` da tabela Drone.

```
INSERT INTO Drone (modelo, autonomia_minutos, alcance_km,
peso)
VALUES
('DR-Alpha', 50, 10.5, 2.5),
('DR-Beta', 35, 8.0, 1.8),
('DR-Gamma', 60, 15.0, 3.2);
```

## Parte III – Consultas SQL

Utilize SELECT com filtros e ordenação.

**8.** Liste todos os professores com salário maior que 5.000, ordenando do maior para o menor salário.

```
SELECT *
FROM Professor
WHERE salario > 5000
ORDER BY salario DESC;
```

**9.** Liste os veterinários contratados após 2020.

```
SELECT *
FROM Veterinario
WHERE data_contratacao > '2020-12-31';
```

**10.** Liste os drones com autonomia superior a 40 minutos.

```
SELECT *
FROM Drone
WHERE autonomia_minutos > 40;
```

**11.** Liste os fornos industriais ativos ordenados pela maior temperatura máxima.

```
SELECT *
FROM FornoIndustrial
WHERE ativo = TRUE
ORDER BY temperatura_maxima
DESC;
```

**12.** Exiba apenas fabricante e pressão máxima das caldeiras com pressão acima de 100.

```
SELECT fabricante, pressao_maxima
FROM Caldeira
WHERE pressao_maxima > 100;
```

## Desafio Extra (Opcional)

Adicione pelo menos uma restrição `CHECK` em qualquer tabela criada (ex: salário maior que zero, pressão maior que zero, autonomia positiva, etc.).

*"Boa atividade! Att. Me. Prof. Sergio Souza Novak."*

```sql
-- Criação da tabela

CREATE TABLE peixes (
id TEXT,
area INTEGER,
perimetro DOUBLE PRECISION,
ecentricidade DOUBLE PRECISION,
maior_eixo DOUBLE PRECISION,
menor_eixo DOUBLE PRECISION,
peso INTEGER,
classe INTEGER,
compactacao DOUBLE PRECISION,
rmm DOUBLE PRECISION,
centroid_x DOUBLE PRECISION,
centroid_y DOUBLE PRECISION
);

-- Estatística descritiva
SELECT MAX(peso), MIN(peso), AVG(peso) FROM peixes;

-- Contagem por classe
SELECT classe, COUNT(id)
FROM peixes
GROUP BY classe
ORDER BY classe;

-- Top 10 peixes mais pesados
SELECT id, peso
FROM peixes
ORDER BY peso DESC
LIMIT 10;

-- Classes distintas
SELECT DISTINCT classe
FROM peixes;

-- Peixes pesados da classe 1
SELECT *
FROM peixes
WHERE peso > 1000 AND classe = 1;

-- Ecentricidade entre 0.5 e 0.8
SELECT id, area
FROM peixes
WHERE ecentricidade BETWEEN 0.5 AND 0.8;
```